

20262809 《Linux内核原理与分析》第十周作业

****学号：****20262809

****姓名：****叶丁再（请确保与证书申请姓名一致）

阅读范围：《庖丁解牛 Linux 操作系统分析》第十章、第十一章

****日期：****2026 年 9 月 27 日

****原创作品转载请注明出处：****叶丁再

课程：《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>

一、这两章在讲什么

第十章介绍 KVM 和虚拟机技术：怎样把一台物理机抽象成多个虚拟机，以及 KVM、QEMU 如何配合完成 CPU、内存和 I/O 虚拟化。第十一章介绍 Linux 容器：在共享宿主机内核的前提下，怎样隔离进程能看到的系统资源，并对进程组的资源使用进行管理。

我把两章的关系概括为：虚拟机主要隔离一套虚拟硬件和 Guest OS；容器主要隔离运行在同一个宿主机内核上的进程环境。两者都能承载应用，但隔离边界、资源开销和内核关系不同。下面按阅读中需要澄清的问题记录理解过程。

二、阅读中的问题与解决思路

问题一：KVM 和 QEMU 是不是同一个虚拟机软件？

****梳理结果：****它们分工不同，合起来才组成常见的 QEMU-KVM 虚拟机方案。KVM 是 Linux 内核提供的虚拟化能力，借助处理器的 VT-x/AMD-V 等硬件支持，让虚拟 CPU 在 Guest 模式下运行，并处理 VM Entry、VM Exit 等切换；QEMU 在用户态负责创建和配置虚拟机、提供虚拟设备及设备模拟。KVM 本身不负责模拟完整的网卡、磁盘等设备，需要时会从 KVM 返回到 QEMU 处理 I/O。

可以把一次运行简化为：QEMU 通过 `/dev/kvm` 和 KVM API 配置虚拟机、内存与 vCPU；vCPU 进入 Guest 执行；遇到需要内核或用户态处理的 VM Exit 后，由 KVM 或 QEMU 处理退出原因，再恢复 Guest 运行。书中的 KVM API 示例依次展示了打开 `/dev/kvm`、查询 API/扩展、创建 VM、设置 Guest 内存、创建 vCPU、映射 `kvm_run` 结构，以及循环调用 `KVM_RUN` 并处理退出原因。

问题二：虚拟机内存地址为什么要经过两次转换？

****梳理结果：****Guest OS 认为自己管理的是一台机器，它的进程先用 Guest Virtual Address (GVA)；Guest OS 的页表把 GVA 转成 Guest Physical Address (GPA)。但 GPA 仍是“虚拟机看到的物理地址”，不是宿主机的真实物理地址。虚拟化硬件再通过 EPT/NPT 等二阶段地址转换，把 GPA 映射到 Host Physical Address (HPA)。因此整体关系是：

GVA → GPA → HPA

把两层转换分开后，就能理解为什么 Guest 仍能使用自己的页表，同时又不会直接决定宿主机真实物理内存的位置。硬件辅助页表减少了由 VMM 软件维护影子页表的开销；TLB 和大页等机制则影响地址转换的效率。

问题三：vCPU 是一颗真实 CPU 吗？谁来调度它？

****梳理结果：****vCPU 是虚拟机看到的处理器抽象。在 KVM 的实现中，vCPU 对应宿主机上的可调度执行线程，仍由 Linux 进程调度器分配物理 CPU 时间。因此，虚拟机调度和宿主机调度是不同层次：Guest OS 可以在自己的 vCPU 上调度 Guest 进程；宿主机 Linux 则调度承载 vCPU 的线程。超分配 vCPU 时，Guest 配置的 vCPU 数可以多于宿主机物理 CPU 数，多个 vCPU 线程需要共享宿主机的 CPU 时间。

这也把本周阅读和前几周的进程调度实验联系起来：无论普通进程还是 vCPU 承载线程，最终都在宿主机的调度机制下获得 CPU；但 Guest 内部还有自己的一层进程调度。

问题四：容器是不是一台更轻的虚拟机？chroot 是不是容器？

****梳理结果：****容器不是完整的虚拟机。容器中的进程与宿主机共享 Linux 内核，不需要为每个容器再启动一套 Guest OS；它依靠 namespace 隔离资源视图，并可用 cgroups 管理资源。这样的结构通常比完整虚拟机轻，但容器中的程序需要兼容宿主机内核和硬件架构。

chroot 只改变当前进程进行路径查找时使用的根目录，能限制它通过该根路径看到的文件范围；它本身不会隔离 PID、网络、挂载树或 CPU/内存资源，也不等同于完整的安全边界。Mount namespace 为进程组提供独立的挂载点视图，隔离范围比只改变根目录更完整。由此我把容器理解为多项内核隔离机制和用户态运行时的组合，而不是一个 chroot 命令。

问题五：namespace 和 cgroups 都是“隔离”，职责有什么区别？

****梳理结果：****namespace 主要回答“进程能看到什么”；cgroups 主要回答“进程组能使用多少资源，以及资源使用如何被记录和控制”。教材列举的 namespace 包括：

类型	主要隔离的视图或资源
Mount	挂载点和文件系统层次
PID	进程号和进程列表
IPC	System V IPC、POSIX 消息队列等
UTS	主机名、域名等系统标识
Network	网络设备、地址、路由和防火墙等
User	用户 ID、组 ID 及相关权限映射

创建 namespace 常见的接口包括 clone()、unshare() 和 setns()：可以在创建子进程时加入新的 namespace，也可以让当前进程脱离原 namespace，或加入已有 namespace。cgroups 则通过层级组织进程组，并由 CPU、memory、blkio 等子系统控制或统计 CPU、内存、块 I/O 等资源。两类机制可以一起使用，但不能互相替代。书中的命令示例采用 cgroups v1 的层级挂载方式；我不把这些具体路径当作所有现代发行版的通用操作步骤。

问题六：OCI 镜像、OCI 包和正在运行的容器有什么区别？

****梳理结果：****OCI 主要定义可互操作的镜像和容器运行规范。教材把 OCI image 描述为由多层文件系统、清单和配置等内容组成的镜像；运行时使用的 OCI bundle 则包含 root filesystem 和 config.json 等配置。镜像是可分发、可复用的应用文件系统及元数据；容器是运行时依据配置创建出来的进程环境。镜像不是正在运行的容器本身。

Docker 把这些底层能力包装成用户更容易使用的一套工具链，包括镜像构建与分发、容器创建和运行。Dockerfile 描述如何构建镜像，镜像仓库用于保存和分享镜像，运行时再依据镜像和配置创建容器。书中提到 Docker 早期使用 LXC，后来封装 namespace、cgroups 等内核能力，减少了对 LXC 的直接依赖。

问题七：StratoVirt 和 iSula 在这张技术图里处于什么位置？

****梳理结果：** **第十章中的 StratoVirt 是基于 KVM 的轻量级虚拟化方案，面向微型虚拟机及标准虚拟机场景；它仍然提供虚拟机层面的隔离。第十一章中的 iSula 是容器软件栈，包含 iSulad 容器引擎，提供 CLI 和 CRI 等接口，并可接入不同的镜像和运行时组件。它与 Docker 一样建立在 Linux 容器相关内核能力之上。

这两个例子让我看到“轻量”并不意味着它们属于同一种机制：StratoVirt 优化的是虚拟机的启动、资源占用和设备模型；iSula 优化的是容器引擎及其运行时栈。判断技术类别时，我会先看隔离边界和执行模型，而不只看启动速度或项目名称。

三、虚拟机与容器的对照

对比点	KVM 虚拟机	Linux 容器
隔离对象	虚拟 CPU、内存、设备及 Guest OS	进程的资源视图和资源使用
内核关系	Guest OS 有自己的内核，KVM 借助宿主机内核和硬件虚拟化运行	容器进程共享宿主机 Linux 内核
关键机制	硬件辅助 CPU/内存虚拟化、设备模拟或虚拟 I/O	namespaces、cgroups、root filesystem 和容器运行时
典型取舍	隔离边界较完整，可运行独立 Guest OS，整体资源开销较高	启动和资源开销通常较低，但依赖宿主机内核兼容性

这个表是概念层面的对照。实际隔离强度还取决于配置、权限、设备暴露和其他安全机制；不能仅凭“用了容器”就推断安全配置已经完善。

四、我的理解

第十章和第十一章分别从硬件层和操作系统层展示了虚拟化。KVM 把 CPU、内存和设备组织成 Guest 能使用的虚拟硬件，QEMU 等用户态组件负责虚拟机管理与设备模型；容器则不再虚构一台完整计算机，而是用 namespace 改变进程看到的系统视图，用 cgroups 管理一组进程的资源，再由 OCI 兼容的运行时和上层工具负责创建、打包和管理。

阅读中的主要收获，是把“虚拟机”“容器”“镜像”“运行时”这些容易混在一起的词，分别对应到隔离边界、内核机制和用户态工具链。它们最终仍依赖 Linux 的进程、内存、文件系统和调度机制；虚拟化并没有让这些基础机制消失，而是在它们之上增加了一层抽象和管理。

阅读范围与说明

本文根据《庖丁解牛 Linux 操作系统分析》第十章“KVM 及虚拟机技术”(教材页 291–308) 和第十一章“Linux 容器技术”(教材页 309–340) 整理。本文是教材预习阅读总结，没有把书中的命令示例写成已在课程云端执行的实验结果，也没有附实验截图。

参考资料

1. 孟宁、娄嘉鹏：《庖丁解牛 Linux 操作系统分析》，第十章“KVM 及虚拟机技术”，第 291–308 页；第十一章“Linux 容器技术”，第 309–340 页。
2. 《Linux 内核分析》MOOC 课程：<http://mooc.study.163.com/course/USTC-1000029000>